

Ch-18|ESP32 Microcontroller

18.1 Xtensa LX6 Microprocessor

In around 2010, Tensilica Company of USA developed a 32-bit customizable (re-configurable) microprocessor, and it was named **Xtensa LX** (Fig-18.1). It is not a fixed function MPU like 8086; rather, it contains many extra hardware blocks (marked as ... and Optional Functional Units in Fig-18.2) which can be activated at product design time to add more functions to the basic Xtensa.



Figure-18.1(a): Xtensa LX MPU

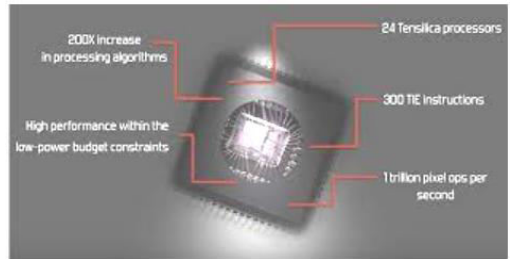


Figure-18.1(b): Xtensa LX MPU

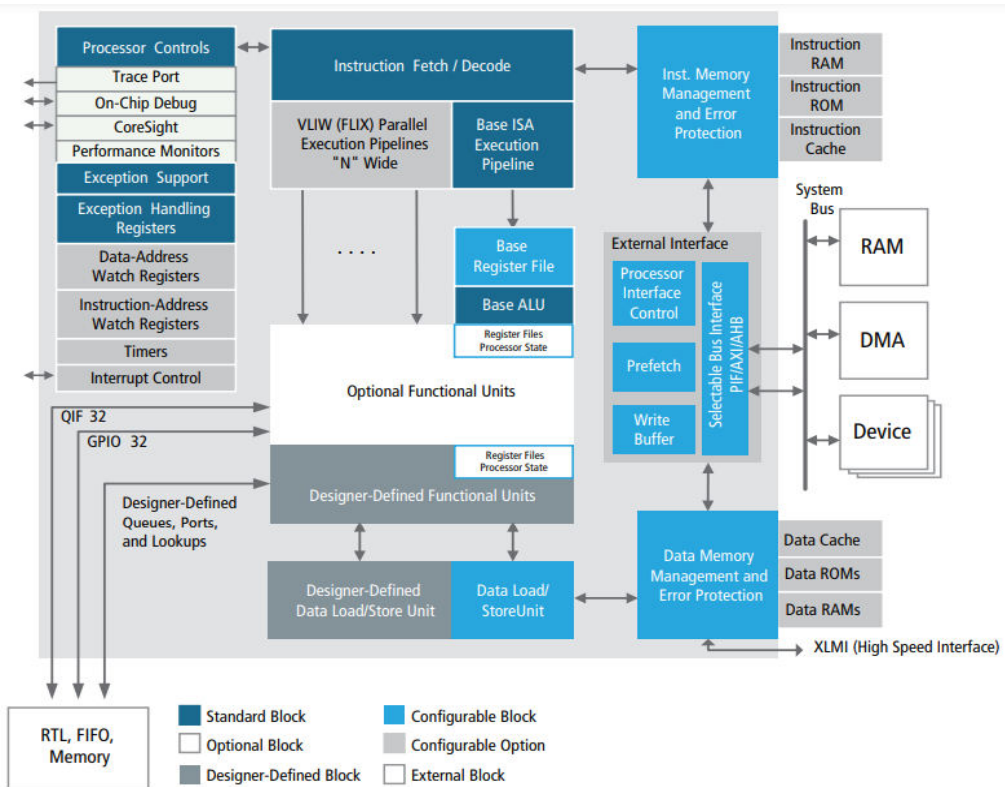


Figure-18.2: Xtensa LX6 Microprocessor showing optional and designer-defined blocks

18.2 ESP32 Microcontroller

In 2013, **Cadence Design Systems** of USA acquired Tensilica. In 2016, **Espressif Systems** Company of China borrowed the MPU from Cadence and built the ESP32 Microcontroller (MCU) chip (Fig-18.3).

The MCU has 48 pins and some pins have one or more signals/functions; whereas, Fig-18.4 shows the default pin signals/functions. For the example of alternate functions, see Section-18.9.

The MCU contains the following resources within the same 48-pin package:

- (1) Two pieces (Cores) of LX6 MPU,
- (2) 440 KB Mask ROM,
- (3) 520 KB RAM,
- (4) 8 KB RTC Fast Memory,
- (5) 8 KB RTC Slow Memory, and
- (6) A large number of peripheral controllers and devices (Fig-18.5).



Figure-18.3: Physical view of 48-pin ESP32 MCU

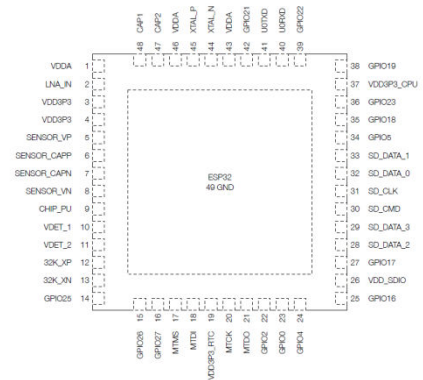


Figure-18.4: Default pin signals of ESP32 MCU

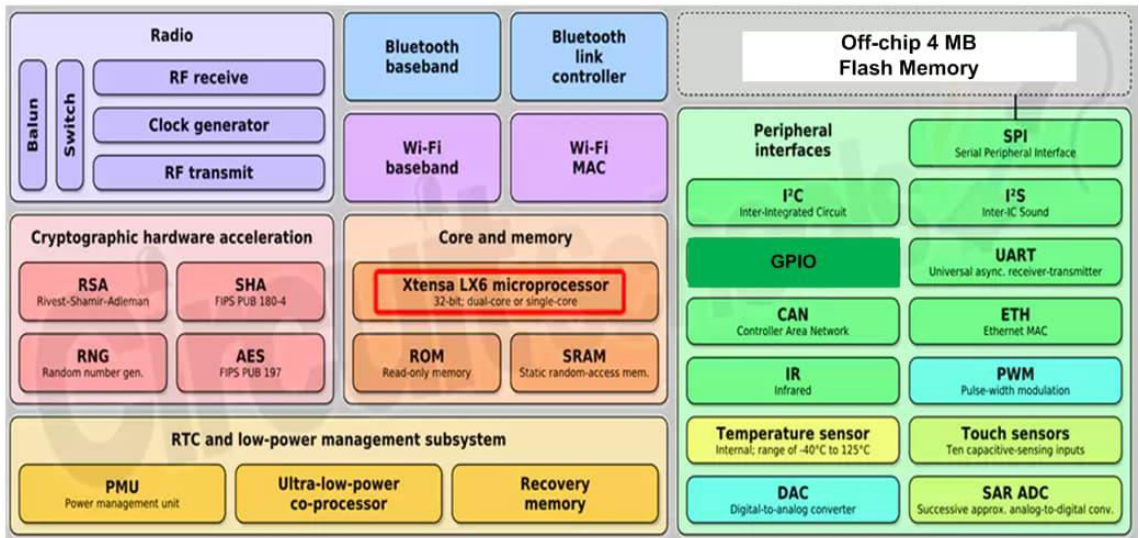


Figure-18.5: Internal peripheral controllers and devices of ESP32

18.3 ESP-WROOM-32 Module

It is a 38-pin small PCB (Fig-18.6) which contains the following hardware resources covered by a Metallic Box/RF Shield. Body of the Metallic Box is a GND-pin, and it is counted as Pin-39.

- (1) ESP32 MCU,
- (2) 48 MHz Crystal for the MCU,
- (3) 4 MB SPI Port based Serial Flash Memory, and
- (4) PCB-based combined antenna for Bluetooth and WiFi.



Figure-18.6:



Figure-18.7:

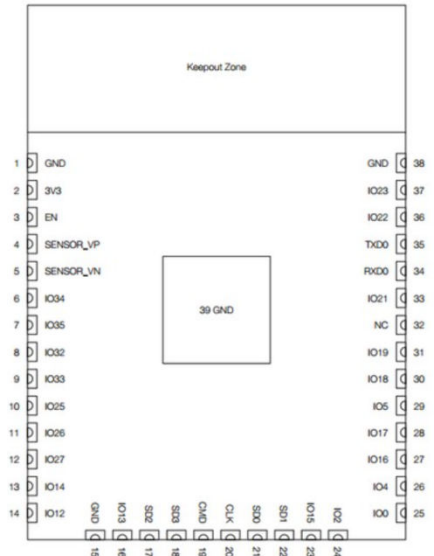


Figure-18.8:

Under the Metallic Cover (Radio Frequency Shield) of Fig-18.6, there are devices like (shown in Fig-18.7): ESP32 MCU, Flash Memory, 40 MHz crystal, and some resistors/capacitors.

The ESP32 has 48 signals/pins, flash memory has 8 signals/pins, and the ESP-WROOM-32 Module (Fig-18.8) has 38 signals/pins. The mappings/connections of pin signals among ESP32, flash, and ESP-WROOM_32 Module are depicted in Fig-18.12.

18.4 ESP32 Dev Module

It is a 30-pin Learning and Development Board (known as ESP32 Dev Module, Fig-18.9). It is built around ESP-WROOM-32 Module. There is only 30-pin in the Module; so, many functions of the root ESP32 MCU (Fig-18.4) are not available to the user. Fig-18.13 of Section-18.6 shows the pin mappings among 48-pin ESP32 MCU, 38-pin ESP-WROOM-32 Module, and 30-pin ESP32 Dev Module.

Once the Board is installed and ESP32 Dev Module is selected, the following Libraries are automatically included in the Arduino IDE. Sketch can be created/uploaded like Arduino UNO.

- (1) ESP-IDF (IoT Development Framework) Library from Espressif Company
- (2) FreeRTOS Library from Real Time Engineers Ltd.

48-pin ESP32 Microcontroller (Core-1)/ESP32 Dev Module-1

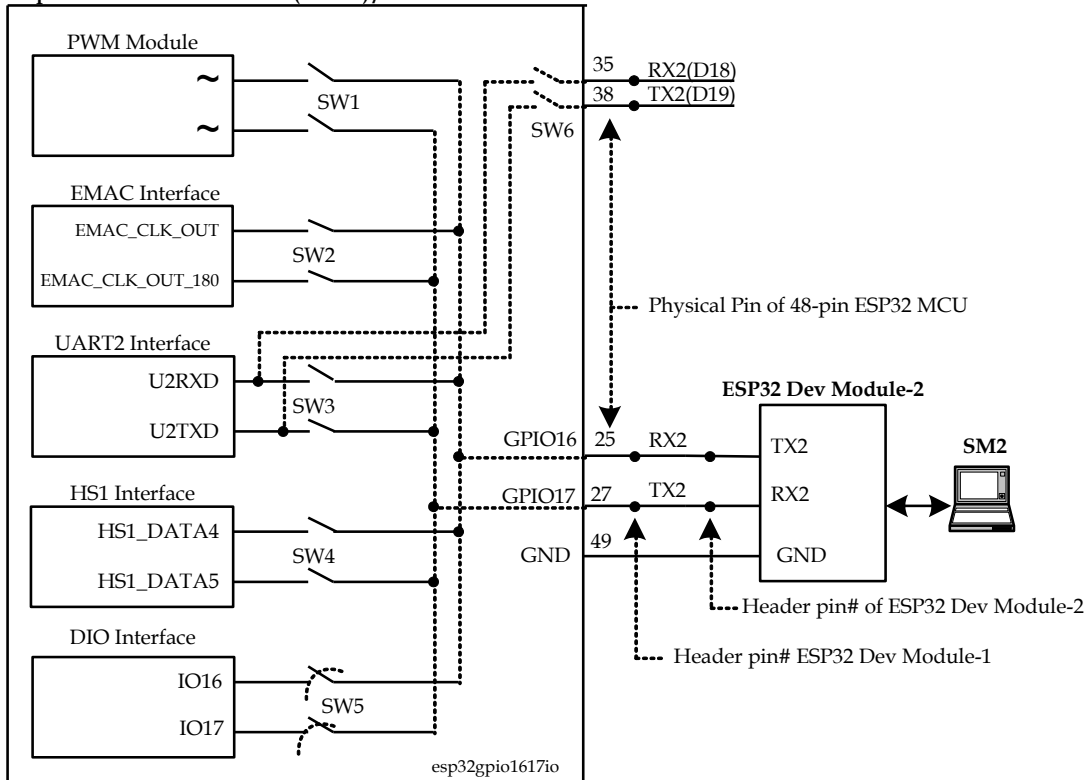


Figure-18.11:

(1) By default, SW5 is closed; as a result, digital IO lines (IO16 and IO17) are connected with Pin-25 and Pin-27 as GPIO16 and GPIO17. These two signals have appeared as D16 and D17 on the header pins of ESP32 Dev Module (Fig-18.8). Now if we connect a LED with D16-pin of Fig-18.8 and upload the following sketch, the LED will turn ON.

```
Void setup()
{
    pinMode(D16, OUTPUT);
    digitalWrite(D16, HIGH);
}
```

```
Void loop(){}

```

(2) If the following code is executed, SW5 of Fig-18.10 will be opened and SW3 will be closed; as a result, D16/D17 pins of ESP32 Dev Module (Fig-18.8) will be working as UART2 Port.

```
Serial2.begin(9600);
```

(3) If we wish that D16/D17 of ESP32 Dev Module will work as digital IO lines and UART2 Interface's signal be routed to other free digital pines (say, D18/RX2 and D19/TX2), then we may execute the following codes. The SW5 remains closed, SW3 remains open, and SW6 gets closed.

```
Serial2.begin(115200, SERIAL_8N1, 18, 19); //Bd, Character=8-bit, No-parity, 1-Stop bit, RX2, TX2
```

(4) Likewise, Serial1 UART1 Interface can be routed to free DPins like D5(RX1) and D18(TX1). The code is:

```
Serial1.begin(115200, SERIAL_8N1, 5, 18); //Bd, Character=8-bit, No-parity, 1-Stop bit, RX1, TX1
```

(5) UART1 Interface can even be routed to D1(RX0) and D5(TX0) for quick check of the functioning of UART1 Port by exchanging message with Serial Monitor which is connected with UART0 Port (Fig-18.11). The code is:

```
Serial1.begin(115200, SERIAL_8N1, 3, 1); //Bd, Character=8-bit, No-parity, 1-Stop bit, RX1, TX1
```

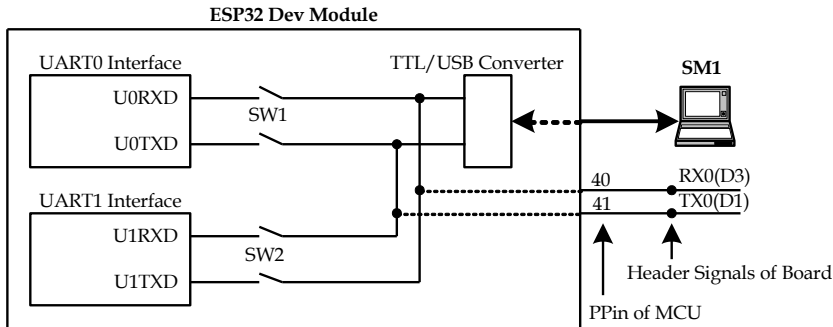


Figure-18.12:

Test Sketch:

```
void setup()
{
  Serial1.begin(115200, SERIAL_8N1, 3, 1); //RX0, TX0
  delay(100);
  Serial1.println("Hello frtom Serial1 !!");
}

void loop()
{
  delay(1000);
  Serial1.println("and again ..");
}
```

When the above sketch is executed, SW1 gets opened and SW2 gets closed. Now, the signals of UART1 Module are connected with Serial Monitor via TTL/USB Converted depicted in Fig-18.11.

18.6 Pin Mapping among 0-pin ESP32 Dev Module, 38-pin ESP-WROOM-32 Module and ESP32 MCU

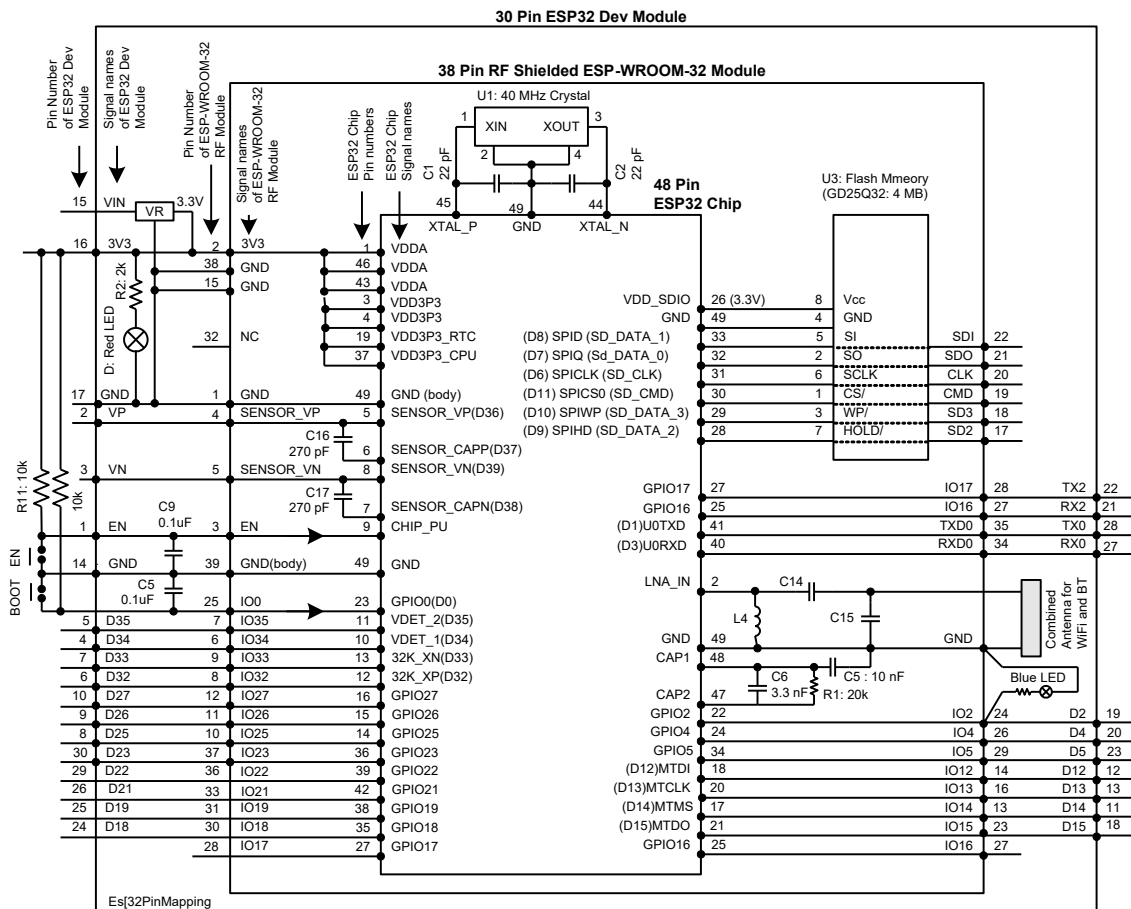


Figure-18.13:

18.7 Memory Structure of ESP32 Microcontroller

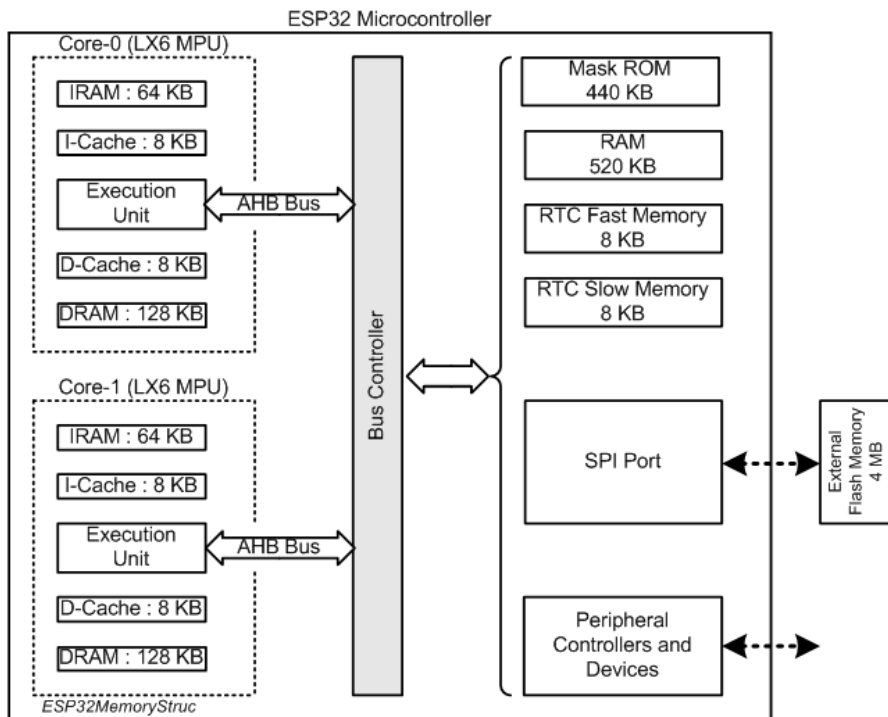


Figure-18.14:

18.8 Programming Examples using ESP32 Dev Module

18.8.1 A tested sketch to know the capacity (in byte) of the off-chip flash memory. The sketch uses programming constructs from ESP-IDF (IoT Development Framework) Library.

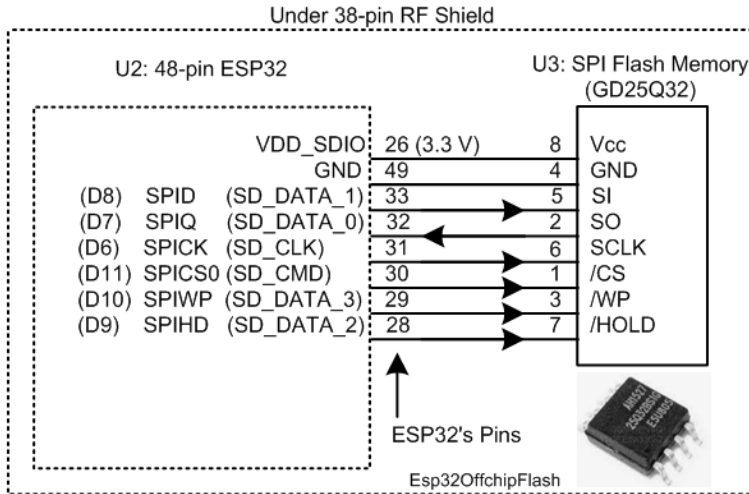


Figure-18.15:

```
#include "esp_system.h"
#include "esp_spi_flash.h"

void setup()
{
  Serial.begin(9600);
  while(!Serial)
  {
    ;
  }
  Serial.println();
  esp_chip_info_t chip_info;
  esp_chip_info(&chip_info); //gives: chip_info.model = 1, chip_info.revision = 3
  uint32_t flashSize = spi_flash_get_chip_size();
  Serial.print("Off-chip Flash Memory Size: ");
  Serial.print(flashSize/(1024*1024)); Serial.println(" MB");
}

void loop() {}
```

18.8.2 Concurrent (independent) Blinking of Three LEDs

LED (onboard) connected at GPIO-2 will be blinking at 4-sec interval (2-sec On and 2-sec Off).

LED10 connected at GPIO-21 will be blinking at 2-sec interval (1-sec On and 1-sec Off).

LED11 connected at GPIO-22 will be blinking at 1-sec interval (0.5-sec On and 0.5-sec Off).

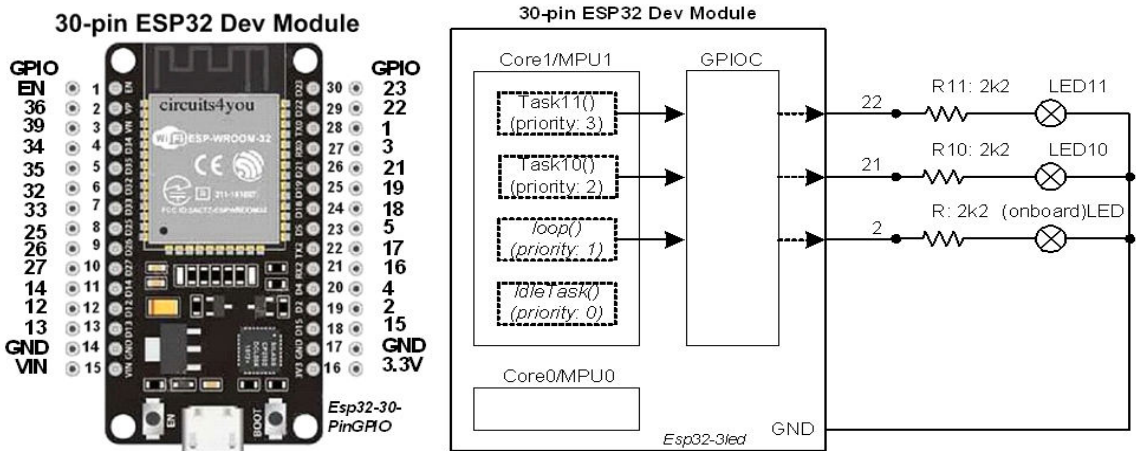


Figure-18.16:

After the uploading of the following sketch, it is observed that the above three LEDs are blinking independently (concurrently) at their respective rates. There is only one MCU which is driving three LEDs one after another in a sequential manner. But, the switching from one LED to the next LED is so fast that they appear to be blinking simultaneously. This fast switching arrangement is carried by FreeRTOS (Real Time Operating System) which is automatically loaded into the ESP32's memory along with the sketch during uploading. There is no timing synchronization among the LEDs. In Section-18.8.3, the concurrent blinking of two LEDs has been shown using *millis()* function and *Arduino UNO*.

```
//function declarations
TaskHandle_t Task10Handle;
TaskHandle_t Task11Handle;

//GPIO definitions
#define LED 2
#define LED10 21
#define LED11 22

//task creation using FreeRTOS.h Librray functions
void setup()
{
    Serial.begin(9600);
    pinMode(LED, OUTPUT);
    pinMode(LED10, OUTPUT);
    pinMode(LED11, OUTPUT);

    xTaskCreatePinnedToCore(Task10, "Task-10", 2048, NULL, 1, &Task10Handle, 1);
    xTaskCreatePinnedToCore(Task11, "Task-11", 2048, NULL, 1, &Task10Handle, 1);
}

//LED blinks at 4000 ms interval
```

```

void loop()
{
    digitalWrite(LED, HIGH);
    vTaskDelay(2000);
    digitalWrite(LED, LOW);
    vTaskDelay(2000);
}
//LED10 blinks at 2000 ms interval
void Task10(void *pvParameters)
{
    while (true)
    {
        digitalWrite(LED10, HIGH);
        vTaskDelay(1000);
        digitalWrite(LED10, LOW);
        vTaskDelay(1000);
    }
}

//LED11 blinks at 1000 ms interval
void Task11(void *pvParameters)
{
    while (true)
    {
        digitalWrite(LED11, HIGH);
        vTaskDelay(500);
        digitalWrite(LED11, LOW);
        vTaskDelay(500);
    }
}

```

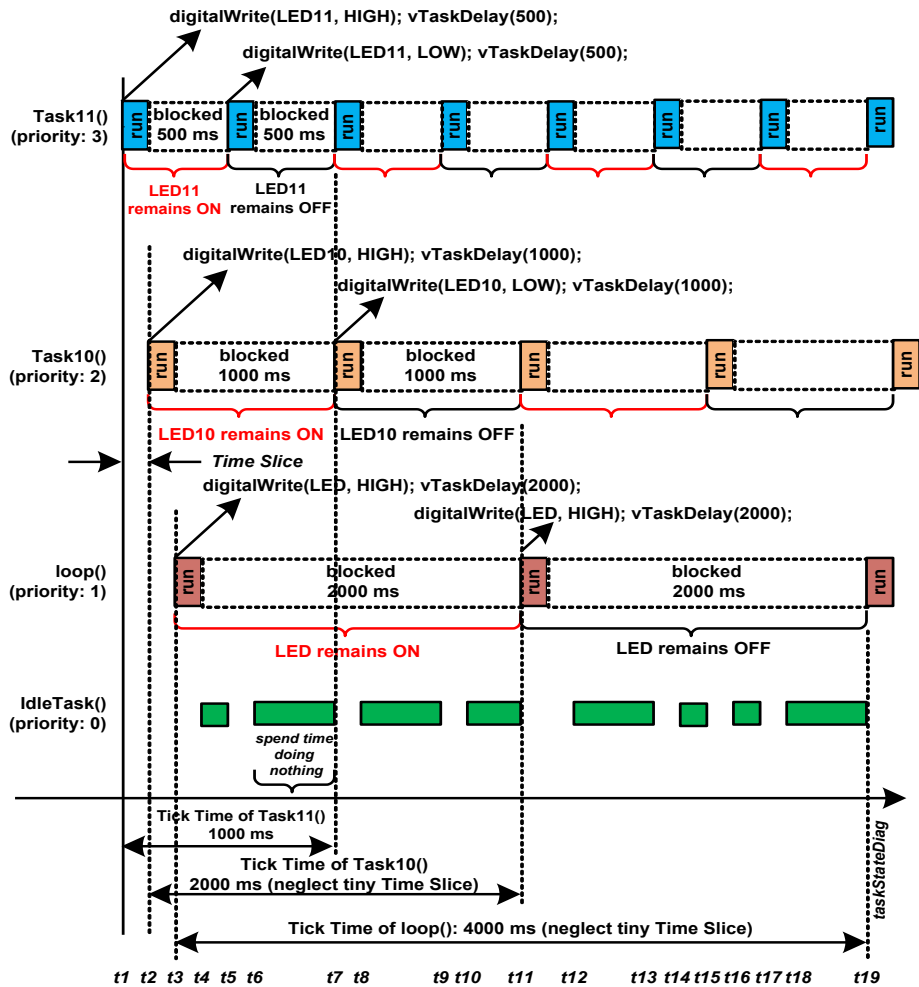


Figure-18.17: Task state diagram for Fig-18.16

(1) Execution of Task10()

At time $t1$ of Fig-18.17, the MPU1 executes the indicated code (*digitalWrite(LED11, HIGH)*) to turn On LED11. Why it is LED11 and NOT LED10? When looking into the sketch, it is seen that Task11() (it drives LED11) has been created with higher priority (3); while, Task10() (it drives LED10), loop() and IdleTask() have lower priorities. So, Task11() is executed first.

After that the *vTaskDelay(250)* code is executed; as a result, the Task11() enters into *blocked* state and marked as *unready* in the *Ready List* of the *Scheduler* (the Supervisory software that manages orderly execution of the Tasks as planned/scheduled).

A Timer (there are four Timers inside ESP32) is started to count the elapsed time. At time $t5$ (when 500 ms has elapsed), with the help of a sophisticated mechanism (vectored interrupt or polled interrupt), the Task11() is marked as *Ready* in the Scheduler's *Ready List*.

The MPU/Scheduler finds that Task11() is ready to run, and it executes the indicated code (*digitalwrite(LED11, LOW)* to turn Off LED11. The immediate execution of *vTaskDelay(250)* puts Task11() into blocked state. At time *t7*, the LED11 will be turned On again.

The turn On, blocked state, turn Off, and blocked state of Task10() (it drives LED10) can be described exactly in the similar way the Task11() has been described above.

(2) Task Switching from Task11() to Task10() (Context Switching)

At the expiry of "Time Slice" period, Task11() has entered into ****Blocked State**** at time *t2* meaning that Task11() is marked as "unready" in the Scheduler's "Ready List".

The Scheduler checks the "Ready List" and finds that Task10() (recall that all Tasks are in ready states when they are created in the *setup()* function) is ready for execution. The MPU executes the indicated codes to turn On LED10. However, the following events occur before executing the very first instruction at time *t2*:

(a) When Task11() was being executed at time *t1*, the MPU was wholly owned by Task11(). The phrase "wholly owned" refers to the fact that the MPU was using its internal "Register Set" and "Program Counter", etc. to fetch/execute the instructions of Task11().

(b) Because only a tiny part of the program (the Task11()) has been executed, the CPU has produced some intermediate results and statuses (which are now within the registers) must be preserved in order to use them when Task11() will come again in execution phase. Also, the (return) address of the next instruction (to be executed when Task11() comes into execution phase) must also be preserved.

(c) Before control is transferred to Task10(), the Scheduler/MPU saves the intermediate results, statuses, and return address onto *stack* memory.

(d) After switching, Task10() owns the MPU and its all resources. The registers and program counter are loaded with a different set of values that are relevant to begin the execution of Task10().

(3) Execution of loop()

(4) Execution of IdleTask()

In Fig-18.17, it is observed Task11() has entered into blocked state at time *t2*; Task110() has entered into blocked state at time *t3*; loop() task has entered into blocked state at time *t4*.

The Scheduler checks the Ready List and finds no ready task to execute. What will it do now?

The MPU must do something which is, in the language of FreeRTOS, the creation of an "Idle Task" (IdleTask()) with priority 0 (the most lowest) for execution.

The CPU keeps executing this IdleTask() (spends CPU cycles doing nothing which is same as executing *nop* (no operation) instruction) until it finds a ready task at time *t5*.

1.8.3 Concurrent Blinking of Two LEDs using Arduino UNO and millis() Function

```
#define LED11 4    //4 sec interval
#define LED12 5    //2 sec interval

unsigned long LED11Millis = millis();
byte LED11State = LOW;
unsigned long LED12Millis = millis();
byte LED12State = LOW;

void setup()
{
  pinMode(LED11, OUTPUT);
  pinMode(LED12, OUTPUT);
  digitalWrite(LED11, LED11State);
  digitalWrite(LED12, LED12State);
}

void loop()
{
  if (millis() - LED11Millis >= 2000)
  {
    LED11State = !LED11State;
    digitalWrite(LED11, LED11State);
    LED11Millis = millis();
  }

  if (millis() - LED12Millis >= 1000)
  {
    LED12State = !LED12State;
    digitalWrite(LED12, LED12State);
    LED12Millis = millis();
  }
}
```

18.19 Problems and Solutions

1 Which GPIO pins support internal pull-up and pull-down resistors?

The following pins supports internal-pull ups:

GPIO2 (often labeled as "D2" on development boards) - Supports both internal pull-up and pull-down.

GPIO4 supports both internal pull-up and pull-down resistors.

GPIO5 supports both internal pull-up and pull-down resistors.

GPIO12 supports both internal pull-up and pull-down resistors.

GPIO13 supports both internal pull-up and pull-down resistors.

GPIO14 supports both internal pull-up and pull-down resistors.

GPIO15 supports both internal pull-up and pull-down resistors.

GPIO25 supports internal pull-up resistor.

GPIO26 supports internal pull-up resistor.

GPIO27 supports internal pull-up resistor.

2 What is the code to activate internal pull-down resistor for the GPIO of ESP32 Board?

```
pinMode(15, INPUT_PULLDOWN); // Activate internal pull-down resistor on GPIO15
```

- 3 Create sketch for the verification of the following task state diagram.

